

Объектно-ориентированное программирование

Лекция 7: Магические методы и перегрузка операторов

Обзор ключевых магических методов (`__str__` , `__repr__` , `__eq__` , `__hash__` , `__len__` , `__call__`). Создание классов-контейнеров. Перегрузка арифметических операций и сравнений для пользовательских объектов.

Магические методы (dunder-методы) в Python — это специальные методы, которые начинаются и заканчиваются двойным подчеркиванием.

Магические методы не предназначены для непосредственного вызова разработчиком, но вызов происходит внутри класса при определенном действии.

Конструкторы и деструкторы	Описание
<code>__new__(cls, other)</code>	Вызывается при создании объекта
<code>__init__(self, other)</code>	Вызывается при создании объекта, но после <code>__new__</code>
<code>__del__(self)</code>	Деструктор

```
1 class Vector:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6 v1 = Vector(2, 3)
7 print(v1)
8 # <__main__.Vector object at 0x...>
```

```
1 class Vector:
2     ...
3     def __repr__(self):
4         return f"Vector({self.x}, {self.y})"
5
6     def __str__(self):
7         return f"Вектор с координатами ({self.x}, {self.y})"
8
9 v1 = Vector(2, 3)
10 print(repr(v1)) # -> Vector(2, 3)
11 print(str(v1))  # -> Вектор с координатами (2, 3)
12 print(v1)       # -> (вызовет __str__) Вектор с координатами (2, 3)
```

```
1 class Vector:
2     ...
3     def __repr__(self):
4         return f"Vector({self.x}, {self.y})"
5
6     def __str__(self):
7         return f"Вектор с координатами ({self.x}, {self.y})"
8
9 v1 = Vector(2, 3)
10 print(repr(v1)) # -> Vector(2, 3)
11 print(str(v1))  # -> Вектор с координатами (2, 3)
12 print(v1)       # -> (вызовет __str__) Вектор с координатами (2, 3)
```

```
1 class Vector:
2     ...
3     def __repr__(self):
4         return f"Vector({self.x}, {self.y})"
5
6     def __str__(self):
7         return f"Вектор с координатами ({self.x}, {self.y})"
8
9 v1 = Vector(2, 3)
10 print(repr(v1)) # -> Vector(2, 3)
11 print(str(v1))  # -> Вектор с координатами (2, 3)
12 print(v1)       # -> (вызовет __str__) Вектор с координатами (2, 3)
```



- `__repr__(self)` — информационная строка об объекте. Выводится при вызове функции `repr( ... )` или в момент отладки. Для последнего этот метод и предназначен.
- `__str__(self)` — вызывается при вызове функции `str( ... )` или `print( ... )`, возвращает строковый объект.
- `__bytes__(self)` — аналогично `__str__(self)`, только возвращается набор байт.
- `__format__(self, format_spec)` — вызывается при вызове функции `format( ... )` и используется для форматирования строки с использованием строковых литералов.


```
1 class Vector:
2     ...
3
4
5 v1 = Vector(2, 3)
6 v2 = Vector(3, 4)
7 v3 = v1 + v2
8 # TypeError: unsupported operand type(s) for +: 'Vector' and 'Vector'
```

```
1  class Vector:
2      ...
3
4      def __add__(self, other):
5          if not isinstance(other, Vector):
6              return NotImplemented
7          return Vector(self.x + other.x, self.y + other.y)
8
9
10 v1 = Vector(2, 3)
11 v2 = Vector(3, 4)
12 v3 = v1 + v2
13 print(v3) # -> Вектор с координатами (5, 7)
```

```
1 class Vector:
2     ...
3
4     def __add__(self, other):
5         if not isinstance(other, Vector):
6             return NotImplemented
7         return Vector(self.x + other.x, self.y + other.y)
8
9
10 v1 = Vector(2, 3)
11 v2 = Vector(3, 4)
12 v3 = v1 + v2
13 print(v3) # -> Вектор с координатами (5, 7)
```

```
1 class Vector:
2     ...
3
4     def __add__(self, other):
5         if not isinstance(other, Vector):
6             return NotImplemented
7         return Vector(self.x + other.x, self.y + other.y)
8
9
10 v1 = Vector(2, 3)
11 v2 = Vector(3, 4)
12 v3 = v1 + v2
13 print(v3) # -> Вектор с координатами (5, 7)
```

```
1  class Vector:
2      ...
3
4      def __add__(self, other):
5          if not isinstance(other, Vector):
6              return NotImplemented
7          return Vector(self.x + other.x, self.y + other.y)
8
9
10 v1 = Vector(2, 3)
11 v2 = Vector(3, 4)
12 v3 = v1 + v2
13 print(v3) # -> Вектор с координатами (5, 7)
```

- `__add__(self, other)` — сложение, оператор `+`
- `__sub__(self, other)` — вычитание, оператор `-`
- `__mul__(self, other)` — умножение, оператор `*`
- `__truediv__(self, other)` — деление, оператор `/`
- `__floordiv__(self, other)` — целочисленное деление, оператор `//`
- `__mod__(self, other)` — остаток от деления, оператор `%`
- `__pow__(self, other[, modulo])` — возведение в степень, оператор `- **`
- `__and__(self, other)` — двоичное И, оператор `&`
- `__xor__(self, other)` — исключающее ИЛИ, оператор `^`
- `__or__(self, other)` — двоичное ИЛИ, оператор `|`

```
1 class Vector:
2     ...
3
4
5 v1 = Vector(2, 3)
6 v2 = Vector(2, 3)
7 print(v1 == v2) # -> False! Сравниваются ссылки на объекты, а не их соде
```

```
1 class Vector:
2     ...
3
4     def __eq__(self, other):
5         if not isinstance(other, Vector):
6             return False
7         return self.x == other.x and self.y == other.y
8
9
10 v1 = Vector(2, 3)
11 v2 = Vector(2, 3)
12 print(v1 == v2) # -> True
```



```
1 class Vector:
2     ...
3
4     def __eq__(self, other):
5         if not isinstance(other, Vector):
6             return False
7         return self.x == other.x and self.y == other.y
8
9
10 v1 = Vector(2, 3)
11 v2 = Vector(2, 3)
12 print(v1 == v2) # -> True
```

```
1 class Vector:
2     ...
3
4     def __eq__(self, other):
5         if not isinstance(other, Vector):
6             return False
7         return self.x == other.x and self.y == other.y
8
9
10 v1 = Vector(2, 3)
11 v2 = Vector(2, 3)
12 print(v1 == v2) # -> True
```

```
1 class Vector:
2     ...
3
4     def __eq__(self, other):
5         if not isinstance(other, Vector):
6             return False
7         return self.x == other.x and self.y == other.y
8
9
10 v1 = Vector(2, 3)
11 v2 = Vector(2, 3)
12 print(v1 == v2) # -> True
```

- `__lt__(self, other)` — определяет поведение оператора сравнения «меньше», `<`.
- `__le__(self, other)` — определяет поведение оператора сравнения «меньше или равно», `≤`.
- `__eq__(self, other)` — определяет поведение оператора «равенства», `=`.
- `__ne__(self, other)` — определяет поведение оператора «неравенства», `≠`.
- `__gt__(self, other)` — определяет поведение оператора сравнения «больше», `>`.
- `__ge__(self, other)` — определяет поведение оператора сравнения «больше или равно», `≥`.

Для использования стандартной функции сортировки `sorted` необходимо определить метод `__lt__`

```
1  class Vector:
2      ...
3
4
5  v1 = Vector(3, 4)
6  len(v1) # TypeError
7  v1[0]   # TypeError
```

```
1 class Vector:
2     ...
3
4     def __len__(self):
5         return int((self.x**2 + self.y**2) ** 0.5)
6
7     def __getitem__(self, index):
8         if index == 0: return self.x
9         if index == 1: return self.y
10        raise IndexError("Индекс вектора может быть 0 или 1")
11
12
13 v1 = Vector(3, 4)
14 print(len(v1)) # 5
15 print(v1[0])   # 3
```

```
1 class Vector:
2     ...
3
4     def __len__(self):
5         return int((self.x**2 + self.y**2) ** 0.5)
6
7     def __getitem__(self, index):
8         if index == 0: return self.x
9         if index == 1: return self.y
10        raise IndexError("Индекс вектора может быть 0 или 1")
11
12
13 v1 = Vector(3, 4)
14 print(len(v1)) # 5
15 print(v1[0])   # 3
```

```
1 class Vector:
2     ...
3
4     def __len__(self):
5         return int((self.x**2 + self.y**2) ** 0.5)
6
7     def __getitem__(self, index):
8         if index == 0: return self.x
9         if index == 1: return self.y
10        raise IndexError("Индекс вектора может быть 0 или 1")
11
12
13 v1 = Vector(3, 4)
14 print(len(v1)) # 5
15 print(v1[0])  # 3
```


- `__len__(self)` — вызывается методом `len( ... )` и возвращает количество элементов в последовательности.
- `__getitem__(self, key)` — вызывается при обращении к элементу в последовательности по его ключу (индексу). Метод должен выбрасывать исключение `TypeError`, если используется некорректный тип ключа, `KeyError`, если данному ключу не соответствует ни один элемент в последовательности.
- `__setitem__(self, key, value)` — вызывается при присваивании какого-либо значения элементу в последовательности. Также может выбрасывать исключения `TypeError` и `KeyError`.
- `__iter__(self)` — вызывается методом `iter( ... )` и возвращает итератор последовательности, например, для использования объекта в цикле.
- `__contains__(self, item)` — вызывается при проверке принадлежности элемента к последовательности с помощью `in` или `not in`.



```
1 class Greeter:
2     def __init__(self, greeting):
3         self.greeting = greeting
4
5     def __call__(self, name):
6         print(f"{self.greeting}, {name}!")
7
8 hello = Greeter("Привет")
9 hello("мир") # Привет, мир!
```

Используя данный метод, можно реализовать хранение состояния между вызовами

- `__hash__(self)` — вызывается функцией `hash( ... )` и используется для определения контрольной суммы объекта, чтобы доказать его уникальность. Например, чтобы добавить объект в `set`, `frozenset`, или использовать в качестве ключа в словаре `dict`.
- `__bool__(self)` — вызывается функцией `bool( ... )` и возвращает `True` или `False` в соответствии с реализацией. Если данный метод не реализован в объекте, и объект является какой-либо последовательностью (списком, кортежем и т.д.), вместо него вызывается метод `__len__`.



- `__getattr__(self, name)` — вызывается методом `getattr( ... )` или при обращении к атрибуту объекта через `x.y`, где `x` — объект, а `y` — атрибут.
- `__setattr__(self, name, value)` — вызывается методом `setattr( ... )` или при обращении к атрибуту объекта с последующим определением значения переданного атрибута. Например: `x.y = 1`, где `x` — объект, `y` — атрибут, а `1` — значение атрибута.
- `__delattr__(self, name)` — вызывается методом `delattr( ... )` или при ручном удалении атрибута у объекта с помощью `del x.y`, где `x` — объект, а `y` — атрибут.
- `__dir__(self)` — вызывается методом `dir( ... )` и выводит список доступных атрибутов объекта.

- `__neg__(self)` — определяет поведение для отрицания (`-a`)
- `__pos__(self)` — определяет поведение для унарного плюса (`+a`)
- `__abs__(self)` — определяет поведение для встроенной функции `abs( ... )`
- `__invert__(self)` — определяет поведение для инвертирования оператором `~`

- `__iadd__(self, other)` — сложение с присваиванием, оператор `+=`
- `__isub__(self, other)` — вычитание с присваиванием, оператор `-=`
- `__imul__(self, other)` — умножение с присваиванием, оператор `*=`
- `__itruediv__(self, other)` — деление с присваиванием, оператор `/=`
- `__ifloordiv__(self, other)` — целочисленное деление с присваиванием, оператор `//=`
- `__imod__(self, other)` — остаток от деления с присваиванием, оператор `%=`
- `__ipow__(self, other[, modulo])` — возведение в степень с присваиванием, оператор `**=`
- `__iand__(self, other)` — двоичное И с присваиванием, оператор `&=`
- `__ixor__(self, other)` — исключающее ИЛИ с присваиванием, оператор `^=`
- `__ior__(self, other)` — двоичное ИЛИ с присваиванием, оператор `|=`

В таких языках, как Java и C#, оператор `new` используется для создания нового экземпляра класса.

В Python магический метод `__new__()` неявно вызывается перед `__init__()` методом.

Метод `__new__()` возвращает новый объект, который затем инициализируется с помощью `__init__()`.



```
1  class MyClass:
2      def __new__(cls, other):
3          print('__new__ called')
4          return super().__new__(cls)
5
6      def __init__(self, other):
7          print('__init__ called')
8
9
10 MyClass(1)
11 # __new__ called
12 # __init__ called
```